Name: ______________ Cohort/Batch: ______________ Date: ______________ Score: ______________

RATE LIMITING PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Microservices

TOTAL: 10 QUESTIONS

Q1 What is Rate Limiting and what problem in Microservices does it solve? Threat Model

Q2 What are the core components or concepts in Rate Limiting? Architecture

Q3 How does Rate Limiting compare to its main alternatives? Configuration

Q4 How do you get started with Rate Limiting for a new project? Hardening

Q5 What configuration options most affect Rate Limiting's behavior in production? SSO / IdP

Q6 How does Rate Limiting handle reliability and high availability? Observability

Q7 What observability does Rate Limiting provide out of the box? Lifecycle

Q8 What are common performance bottlenecks in Rate Limiting? Performance

Q9 What security best practices apply when running Rate Limiting? Testing

Q10 What mistakes do teams commonly make when adopting Rate Limiting? Tuning

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Rate Limiting is a tool or platform

Mitigation

Rate Limiting is a tool or platform that automates or simplifies a specific concern in the Microservices domain, reducing manual effort and operational complexity for engineering teams.

A2 Rate Limiting has key building blocks that

Primitives

Rate Limiting has key building blocks that work together: a configuration layer defines intent, a runtime layer enforces it, and an API or CLI layer allows operators to manage and observe the system.

A3 Rate Limiting trades off simplicity against feature

Hardening

Rate Limiting trades off simplicity against feature richness differently than its alternatives. Choose Rate Limiting when its specific strengths performance, ecosystem, or ease of use align with your team's primary constraints.

A4 Install Rate Limiting via its package manager

Gotchas

Install Rate Limiting via its package manager or binary release. Follow the quickstart to initialize a project, configure the minimal required settings, and run the default command to verify the setup works end-to-end.

A5 Production-critical settings typically include concurrency

Federation

Production-critical settings typically include concurrency limits, timeout values, retry policies, resource limits, and logging levels. Review the official production hardening guide before deploying.

A6 Rate Limiting provides HA through leader election,

Observability

Rate Limiting provides HA through leader election, replication, or stateless horizontal scaling. Deploy at least two instances across availability zones and configure health checks for automatic failover.

A7 Rate Limiting exposes metrics (Prometheus endpoint or

Automation

Rate Limiting exposes metrics (Prometheus endpoint or built-in dashboard), structured logs, and optionally distributed traces. Define SLOs on the key metrics and alert before users are affected.

A8 Performance bottlenecks typically stem from misconfigured

Governance

Performance bottlenecks typically stem from misconfigured connection pools, large payload sizes, insufficient worker threads, or missing caches. Profile with built-in diagnostics before optimizing.

A9 Run Rate Limiting with the principle of

Assessment

Run Rate Limiting with the principle of least privilege, enable TLS for all communication, use a secrets manager for credentials, and keep the software patched to the latest stable release.

A10 Common pitfalls include skipping the documentation and

Optimization

Common pitfalls include skipping the documentation and learning the API by trial and error, using default configurations in production, lacking a runbook for operational issues, and not testing failover scenarios.